DA3408 A3 — Spark vs. Ray: The Data Engineering Duel

Benchmark Report: NYC Yellow Taxi Data, 2023–2025

1. Objective

The project implements the same distributed preprocessing pipeline using Apache Spark and Ray Data and compares their execution performance on NYC Yellow Taxi Trip Data. The experiment evaluates both native expression-based computation and a Python UDF implementation of average speed.

2. Dataset and Pipeline

• Dataset period: January 2023 through December 2025 (36 monthly Parquet files).

• Reference lookup: taxi_zone_lookup.csv.

• Ingestion: monthly Yellow Taxi Parquet files.

• Cleansing: select required columns, filter null values, remove duplicates, normalize timestamps, and cast numeric fields.

• Transformation: pickup and drop-off location joins, trip-duration calculation, and average-speed calculation.

• Export: processed results written to Parquet.

3. Cluster Configuration

Framework	Head/Master	Workers	CPU allocation
Spark	1 master	2 workers	5 cores/worker
Ray	1 head	2 workers	4 CPUs/worker + head resources

4. Benchmark Results

Framework	Mode	Total time (s)
Spark	Native	389.903
Spark	Python UDF	444.087
Ray	Native	500.061
Ray	Python UDF	556.561

Spark Native is approximately 110.158 seconds faster than Ray Native (22.03% faster relative to Ray). Spark UDF is approximately 112.474 seconds faster than Ray UDF (20.21% faster relative to Ray).

5. Python UDF Analysis

The Python UDF adds overhead in both frameworks. Spark increases from 389.903 s to 444.087 s, an overhead of 54.184 s (13.9%). Ray increases from 500.061 s to 556.561 s, an overhead of 56.500 s (11.3%). Thus Ray has slightly lower relative UDF overhead, while Spark remains faster in absolute runtime.

6. Pipeline Parity

Metric	Result
Rows	128,193,192
Sum distance	694,895,818.02

Sum fare	2,438,620,410.96
Average duration (hours)	0.286685633
Valid speed count	127,617,817
Average valid speed	19.6269210472
Minimum valid speed	0.0
Maximum valid speed	5,379,164.6
Final parity	PASS

All four pairwise comparisons passed for row count, distance sum, fare sum, average duration, valid-speed count, average speed, minimum speed, and maximum speed. This supports that the Spark and Ray implementations produced equivalent aggregate results.

7. Resource Usage

Container	Peak CPU	Peak memory
Ray head	238.77%	1.333 GiB
Ray worker 1	523.62%	2.439 GiB
Ray worker 2	487.71%	2.189 GiB
Spark master	385.53%	770.1 MiB
Spark worker 1	410.55%	1.469 GiB
Spark worker 2	114.42%	518.5 MiB

These are observed peak values from the resource-monitoring runs, not theoretical maximum capacities.

8. Performance Discussion

• Spark achieved the lowest total wall-clock time in both native and Python-UDF modes.

• The native implementation is faster than the Python-UDF implementation in both systems, demonstrating the cost of Python UDF execution.

• The monthly execution strategy was used to keep individual processing stages manageable and avoid failures observed during larger all-at-once executions.

• The same transformation semantics were retained across Spark and Ray, and the final parity check passed across all four implementations.

9. Performance Tuning Note

AI assistance was used to translate equivalent Spark and Ray pipeline logic and to reason about performance settings. Spark used 80 shuffle partitions and disabled automatic broadcast joins. Ray used 80 blocks/repartitioning. Both pipelines were executed month-by-month to control resource pressure while preserving the same logical workload.

10. AI-first vs. BI-first Conclusion

This workload is primarily BI-first. The main task is structured, large-scale data engineering: ingesting multiple files, cleansing records, joining lookup data, computing derived features, and exporting Parquet data. AI assistance was useful for code translation and tuning, but the workload itself is not centered on model training or inference. Therefore, the benchmark is best characterized as a BI/data-engineering workload with AI-assisted development.

11. Conclusion

The experiment demonstrates that Spark was faster than Ray for this workload under the tested cluster and configuration. Spark completed the native pipeline in 389.903 s and the UDF pipeline in 444.087 s, while Ray required 500.061 s and 556.561 s respectively. Despite the runtime difference, both frameworks produced matching aggregate results, with the final parity check passing for all tested metrics.

Benchmark values and parity statistics are based on the recorded experiment results from the project execution.